

PHASE 8 : PROJECT DOCUMENTATION

TEAM LEAD : KAVIYA.P

**TEAM MEMBERS : FEMINASHI.B ,
HEMADHARSHINI.S ,
JAYANTHARA.V ,
JAYASHREE.S**

PROJECT TITLE : AI FAQ ASSISTANT

COLLEGE NAME : M.G.R COLLEGE,HOSUR

AI FAQ ASSISTANT

1 Introduction

The AI FAQ Assistant is an AI-powered FAQ and customer support automation platform. It is designed as a secure REST API that supports FAQ creation, authentication, content management, search, and AI-assisted generation. The application uses Node.js and Express.js for backend development and MongoDB with Mongoose for data storage and validation. Security is implemented through JWT-based authentication and bcrypt password hashing. The system also integrates Google Gemini AI for automated question and answer generation. Its purpose is to reduce repetitive support work and improve the organization and discoverability of FAQ content. The architecture separates business logic from frontend delivery and provides structured API endpoints for consumers and authenticated creators.

Source basis: AI FAQ Assistant project document. [filecite](#)[turn0file0](#)[L5-L14](#)

2 Problem Statement

The project addresses the difficulty of managing large volumes of repetitive customer questions. Manual FAQ creation can require considerable effort for organizing topics, preparing accurate answers, maintaining documentation, and identifying recurring question trends. The case study describes a support operations manager dealing with backlogs of inquiries and limited visibility into semantic search keywords. Another challenge is the lack of a unified permission layer for private documentation. These problems can lead to slower support workflows and inconsistent information management. The proposed system centralizes FAQ creation, search, authentication, and AI-assisted content generation within a single backend service.

Source basis: AI FAQ Assistant project document. [filecite](#)[turn0file0](#)[L5-L14](#)

3 Objectives

The main objective is to build a centralized and secure backend for FAQ and customer-support automation. The system aims to provide authenticated users with tools for registering accounts, generating FAQs, creating records, modifying their own records, and removing records they own. It also provides public access to published FAQ information and search functionality. Another objective is to use AI to generate questions, answers, and topic suggestions. Data integrity is supported through Mongoose schema validation. Security objectives include JWT route protection, password hashing, authorization checks, and centralized error handling. The overall objective is to make FAQ management structured, searchable, and easier to maintain.

Source basis: AI FAQ Assistant project document. □filecite□turn0file0□L5-L14□

4 Software Requirements

The documented software requirements include Windows 10 or 11, macOS, or Linux as supported operating systems. Node.js version 18 or above provides the runtime environment for the server. npm version 9 or above manages project dependencies. Express.js provides the backend REST routing framework. MongoDB is the NoSQL database, while Mongoose provides object data modeling and schema validation. Postman or ThunderClient can be used for API testing and verification. Visual Studio Code or a similar code editor is suitable for development. The project also uses packages including dotenv, cors, jsonwebtoken, bcrypt, and the Google GenAI package as shown in the setup commands.

Source basis: AI FAQ Assistant project document. □filecite□turn0file0□L5-L14□

5 Hardware Requirements

The documentation specifies an Intel Core i5 eighth generation processor or above, or an equivalent AMD Ryzen 5 processor. The minimum RAM requirement is 8 GB, while 16 GB is recommended when running concurrent MongoDB processing layers. The required available storage is approximately 1 GB of disk workspace. These requirements describe a development environment capable of running Node.js, MongoDB-related services, the code editor, and API testing tools. The hardware requirements are intended to support the backend development and testing workflow rather than define production infrastructure sizing.

Source basis: AI FAQ Assistant project document. [filecite](#)[turn0file0](#)[L5-L14](#)

6 Project Architecture

The AI FAQ Assistant exposes structured API endpoints through a layered server architecture. Incoming requests pass through filtering and middleware before reaching business logic and data services. API tools such as Postman communicate with route paths and receive JSON entities. The Express server gateway monitors network ports, maps HTTP methods such as GET, POST, PUT, and DELETE, parses JSON payloads, and handles CORS headers. Authentication middleware extracts authorization tokens and validates user context. Granular service modules enforce domain rules. The database interface uses Mongoose to translate application operations into MongoDB queries. The AI flow connects to external AI models through Google Gemini SDK integrations.

Source basis: AI FAQ Assistant project document. [filecite](#)[turn0file0](#)[L5-L14](#)

7 Database and ER Model

The backend database uses MongoDB collections and logical relationships to support authentication, FAQ management, AI-generated content, and categories. The User entity contains an ObjectId, name, unique email, hashed password, role, and timestamps. The FAQ entity contains an identifier, question, answer, category, creator reference, and timestamps. AI generation is described as a logical entity created dynamically through Google Gemini before persistence. Category information includes a category name, description, and total FAQ count. A user can manage multiple FAQs, a category can contain multiple FAQs, and a user can generate multiple AI-assisted FAQs. Search results are logically related to matching FAQ documents.

Source basis: AI FAQ Assistant project document. [filecite\turn0file0\L5-L14](#)

8 Key Features

The system provides JWT authentication and authorization to protect private API endpoints. AI FAQ generation uses Google Gemini to automatically generate questions, answers, and topic suggestions. Semantic or keyword-based FAQ search supports faster information retrieval. Complete CRUD operations allow FAQ records to be created, read, updated, and deleted. Category management provides structured organization. Passwords are hashed using bcrypt before storage. Centralized error handling produces standardized responses. Mongoose schema validation protects data integrity before persistence. The RESTful Express architecture provides modular backend services, while MongoDB supplies document-oriented data storage.

Source basis: AI FAQ Assistant project document. [filecite\turn0file0\L5-L14](#)

9 User Roles

The documented roles include Admin, Content Creator, Authenticated User, and Public User. The Admin can manage registered users, view or modify FAQs, monitor AI-generated content, manage categories, and access complete system functionality. A Content Creator can create FAQs, edit personal records, delete owned FAQs, generate FAQs using AI, and organize FAQs into categories. An Authenticated User can view public FAQs, search using keywords, generate AI-assisted answers, and access personal profile information. A Public User can view published FAQs and search FAQs but cannot create, edit, or delete records and cannot access protected API endpoints. These roles establish different permission levels within the system.

Source basis: AI FAQ Assistant project document. [filecite\turn0file0\L5-L14](#)

10 MVC Architecture

The system follows an MVC-oriented architecture adapted for a headless backend. The Model layer defines data blueprints through Mongoose schemas. These schemas specify fields, validation rules, and defaults before records are persisted to MongoDB. The Controller layer acts as the execution matrix. It receives requests from routes, checks request constraints, delegates workflows to models or services, and prepares JSON responses. The View layer does not contain conventional web templates because this is a headless backend. Instead, the API routing layer functions as the interface between network requests and controller handlers. This separation helps organize responsibilities across the application.

Source basis: AI FAQ Assistant project document. [filecite\turn0file0\L5-L14](#)

11 Project Setup

The documented project setup begins by creating a project folder and a src subfolder. The project is opened in Visual Studio Code. The root terminal is used to initialize npm with the command `npm init -y`. Dependencies are installed using commands that include Express, Mongoose, dotenv, cors, jsonwebtoken, bcrypt, and the Google GenAI package. This setup creates the runtime and dependency foundation for the application. The src directory is used for backend source code. A structured folder arrangement helps separate server configuration, application setup, middleware, models, routes, and related logic.

Source basis: AI FAQ Assistant project document. □filecite□turn0file0□L5-L14□

12 Backend Development

The backend development phase contains core system script implementations. The main server file is identified as `src/server.js`. This entry mechanism aggregates environment variables, triggers database listeners, and binds the application to the configured network port. The Express application is represented by `src/app.js`. Security access middleware is implemented in `src/middleware/authMiddleware.js`. This middleware intercepts requests, extracts authorization information, and validates token identity claims. A centralized error handler is located in `src/middleware/errorMiddleware.js`. These components collectively provide the application gateway, security layer, and standardized error-management process.

Source basis: AI FAQ Assistant project document. □filecite□turn0file0□L5-L14□

13 Database Configuration

Database configuration is handled using Mongoose schema definitions. The User model is located at `src/models/User.js` and enforces profile-field criteria and validation. The documentation states that password hashing occurs before the user document is persisted. The FAQ model is located at `src/models/FAQ.js`. It defines core FAQ properties and maps a relational reference to the creator document. This approach allows MongoDB documents to maintain structured relationships through identifiers while preserving the flexibility of a NoSQL database. Schema validation ensures that required fields and expected data structures are checked before persistence.

Source basis: AI FAQ Assistant project document. [filecite](#)[turn0file0](#)[L5-L14](#)

14 Authentication and Security

Security is a central feature of the application. JWT tokens provide stateless user sessions and route guarding. Authentication middleware extracts authorization tokens from request headers and validates their identity context. Passwords are not stored as plain text; the documented implementation uses `bcrypt` hashing before persistence. Protected resources require authentication and authorization. The project also uses centralized error handling and response status sanitization to reduce the risk of exposing database or internal implementation details. These mechanisms work together to control access to private endpoints and maintain separation between public operations and authenticated operations.

Source basis: AI FAQ Assistant project document. [filecite](#)[turn0file0](#)[L5-L14](#)

15 API Routes

The documented API routing interface includes authentication, FAQ, and AI automation routes. Public authentication operations are located in `src/routes/authRoutes.js`. Protected FAQ operations are located in `src/routes/faqRoutes.js`. AI automation operations are located in `src/routes/aiRoutes.js`. The registration endpoint is `POST /api/auth/register` and is public. The login endpoint is `POST /api/auth/login` and is also public. FAQ creation uses `POST /api/faqs` and requires a Bearer JWT token. FAQ search uses `GET /api/faqs/search?q=configure` and is public according to the documentation. AI FAQ generation uses `POST /api/ai/generate-faq` and requires a Bearer JWT token.

Source basis: AI FAQ Assistant project document. [filecite](#)[turn0file0](#)[L5-L14](#)

16 Registration and Login

The registration process uses `POST /api/auth/register`. The example request contains a name, email address, and password. The documented example uses the name Priya Sharma, an email address, and a sample password. After registration, login verification uses `POST /api/auth/login`. The login request contains the email and password. Authentication allows the system to issue or validate a token used for protected operations. The security design separates public onboarding and login routes from protected FAQ and AI routes. This workflow allows users to establish an identity before accessing functionality that requires authorization.

Source basis: AI FAQ Assistant project document. [filecite](#)[turn0file0](#)[L5-L14](#)

17 FAQ Creation

FAQ creation is demonstrated through POST /api/faqs. The endpoint is private and requires a Bearer JWT token. The example request contains a question, an answer, and a category. The example question concerns configuring custom category tags. The answer directs a user toward settings, keyword tags, and saving the schema. The category is Configuration. When a valid authenticated request is submitted, the backend can validate the data through the FAQ model and associate the FAQ with its creator. This provides a structured method for adding support documentation to the system.

Source basis: AI FAQ Assistant project document. □filecite□turn0file0□L5-L14□

18 Search Functionality

The project provides an FAQ search endpoint using GET /api/faqs/search?q=configure. The endpoint is documented as public. Search functionality is intended to improve information retrieval by finding FAQ documents that match query terms. The architecture description also refers to semantic matching and keyword matching. Search results have a logical relationship with FAQ documents. High-performance indexing is identified in the documented outcome as a mechanism supporting real-time discoverability. This feature helps public and authenticated users locate relevant FAQ content without manually reviewing every document.

Source basis: AI FAQ Assistant project document. □filecite□turn0file0□L5-L14□

19 AI FAQ Generation

AI automation is implemented through POST `/api/ai/generate-faq`. The endpoint requires a Bearer JWT token. The request body contains a topic value. The documented example topic concerns Mongoose schema indexing, validation, runtime workflow, and optimization. Google Gemini AI is integrated into the service flow to generate questions, answers, and category suggestions. AI-generated information is described as a logical entity that can later be saved into the FAQ collection. This feature reduces manual drafting effort and supports faster creation of structured support content.

Source basis: AI FAQ Assistant project document. [filecite](#)[turn0file0](#)[L5-L14](#)

20 API Testing and Execution

The project execution phase recommends installing Postman or the ThunderClient VS Code extension. The backend development server is started using `npm run dev`. API testing verifies operational structural integrity by sending requests to documented endpoints. Registration and login requests test identity operations. FAQ creation tests authenticated resource creation. Search tests keyword retrieval. AI generation tests the integration between the backend and the AI service. Testing provides a way to verify that routes, validation, authentication, database operations, and response behavior work as intended. The supplied documentation also includes a testing video and demo link.

Source basis: AI FAQ Assistant project document. [filecite](#)[turn0file0](#)[L5-L14](#)

21 Error Handling and Validation

Centralized error handling is included as a reliability feature. The system is designed to provide standardized API error responses and reduce unhandled database leakage. Mongoose validation ensures that required fields and data structures are checked before persistence. Authentication middleware protects guarded resources. Response status sanitization is described as a measure that helps prevent internal database details from being exposed. Together, validation and error handling improve operational durability. They also make debugging more systematic because errors can be handled through a common middleware mechanism rather than through unrelated responses in individual routes.

Source basis: AI FAQ Assistant project document. [filecite\turn0file0\L5-L14](#)

22 Workflow and Data Flow

A typical workflow begins when a user or API client sends a request to the Express server. The server parses the request and applies middleware. Public requests can continue to available public routes, while protected routes require token validation. Controllers process validated requests and communicate with models or service modules. Mongoose validates and persists data in MongoDB when a database operation is required. For AI generation, the controller or service delegates content generation to the Google Gemini integration. The generated information can be returned to the user and may later be persisted as an FAQ. Search requests retrieve relevant FAQ documents based on matching terms.

Source basis: AI FAQ Assistant project document. [filecite\turn0file0\L5-L14](#)

23 Advantages and Expected Outcome

The documented outcome is reduced operational delay through centralized workflow controls. Data structural integrity is maintained through automated schema validation. Security tokens restrict modification of documents to authorized users and original authors where applicable. Indexing supports data discoverability. The architecture separates business logic from frontend delivery, making the backend usable by different clients. AI assistance can reduce the manual effort required to draft FAQ content. Categorization and search improve organization and retrieval. These outcomes are aligned with the project's purpose of supporting customer service operations through structured documentation and automation.

Source basis: AI FAQ Assistant project document. □filecite□turn0file0□L5-L14□

24 Conclusion

The AI FAQ Assistant is a backend platform for secure FAQ management and AI-powered customer support automation. It combines Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, and Google Gemini integration. Its architecture includes routing, middleware, controllers, models, database services, and AI services. The system supports multiple user roles, CRUD operations, search, category management, authentication, validation, and AI-generated FAQ content. The documented API examples provide a practical basis for testing registration, login, FAQ creation, search, and AI generation. Overall, the project demonstrates how a modular REST API can organize support documentation while adding AI-assisted content generation.

Source basis: AI FAQ Assistant project document. □filecite□turn0file0□L5-L14□

25 References and Project Resources

The primary source for this documentation is the supplied AI FAQ Assistant project document. It contains the project description, requirements, architecture, entities, features, roles, setup commands, implementation structure, API endpoints, testing examples, and project resources. The source also provides an API testing video link and a demonstration link. These resources can be used alongside the documented API specifications when reviewing or demonstrating the project. The documentation should be treated as a structured presentation-style expansion of the supplied project material, while implementation-specific details should be verified against the actual source code when required.

Source basis: AI FAQ Assistant project document. □filecite□turn0file0□L5-L14□